Spring Framework

Spring 02 – Accès aux données avec Spring Data (JPA)

Rappel des objectifs du cours

- **Découvrir les fonctionnalités apportées par Spring Data JPA**
- **Montrer les possibilités de Spring Data JPA**
- **Apprendre à écrire des tests unitaires sur les méthodes du repository**

Sommaire

- **Introduction à Spring Data JPA**
- **Les Repositories / DAO avec Spring Data JPA**
  - *Partie 1 : Utiliser les interfaces Spring*
  - *Partie 2 : Utiliser les requêtes dérivées du nom de méthode*
  - *Partie 3 : Utiliser @Query*
  - *Partie 4 : Implémentations personnalisées*
  - *Partie 5 : Transactions*
- **Tests unitaires sur les Repositories Spring**

Spring Framework

Introduction à Spring Data JPA

Qu'est-ce que JPA ?

- **JPA** est une API qui signifie **J**ava **P**ersistence **A**PI
- **JPA** est une **spécification** définissant des règles et des interfaces pour l'accès aux bases de données relationnelles.
- Cette API a plusieurs implémentations:
  - **Hibernate**
  - Ibatis
  - Eclipse-link
- Pour accéder à une base de données il faut utiliser une implémentation de **JPA**.

Qu'est-ce que l'ORM ?

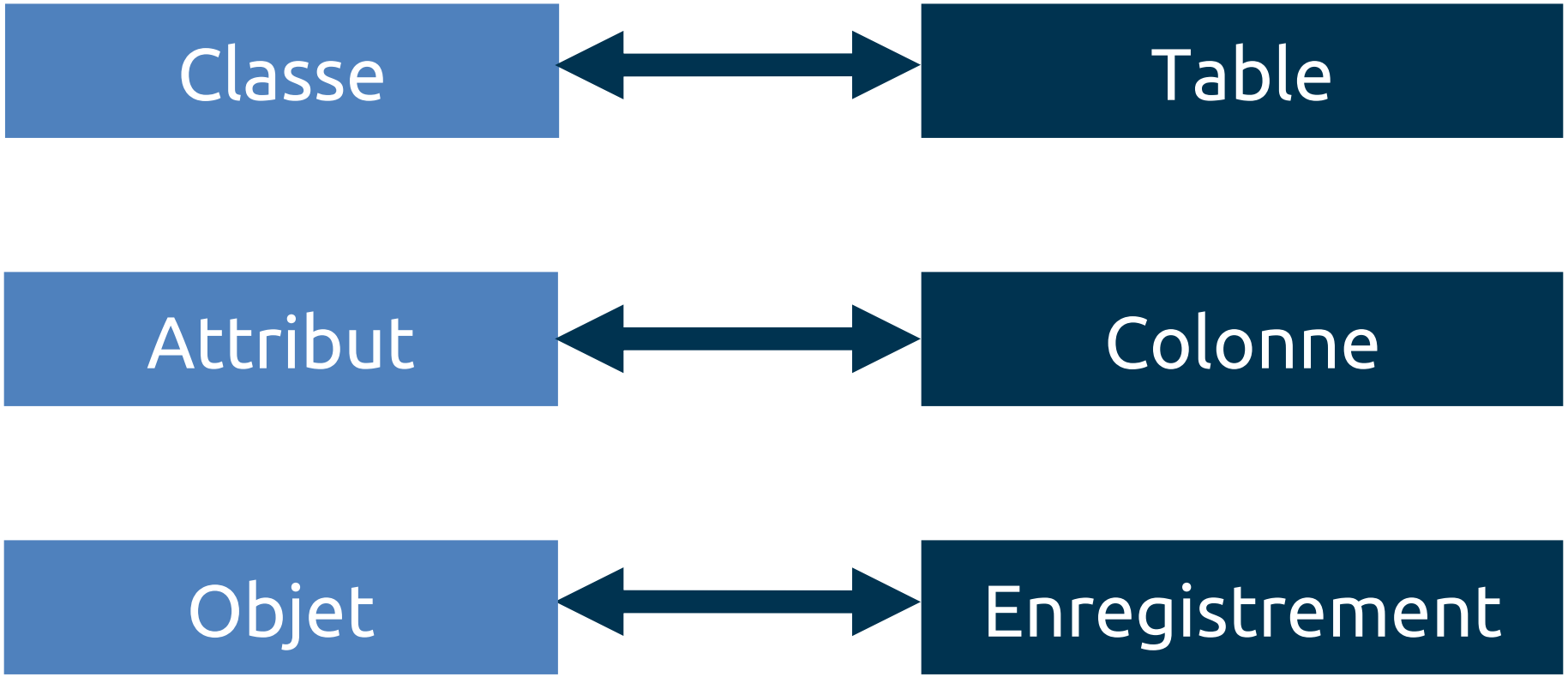
- **JPA** est basé sur le mécanisme d'**ORM** ou **O**bject **R**elational **M**apping
- Le principe de l'**ORM** est de mapper des classes sur des tables.
- L'idée derrière **JPA** est d'écrire le moins de SQL possible et de laisser la librairie utilisée les générer.
- Quelle est la différence entre **JPA** et **ORM** ? L'**ORM** est un principe plus large sur lequel repose **JPA**.

Couche de persistance sans ORM

Le développeur a la possibilité d'écrire manuellement la persistance des objets:

- Avec l'API JDBC – très bas niveau
- Toutes les étapes d’utilisation de JDBC apparaissent clairement dans le code des classes
- La gestion des connexions
- L’écriture manuelle des requêtes
- Plus de dix lignes de codes sont nécessaires pour rendre persistante une classe contenant des propriétés simples
- Développement objets "très couteux"

ORM - Principe



ORM - Exemple

```
public class Personne {
        private int id;
        private String nom;
        private String prenom;
}

Personne p1 = new Personne(1, "Durand", "Paul");
Personne p2 = new Personne(2, "Dupont", "Pierre");
```

Table PERSONNE

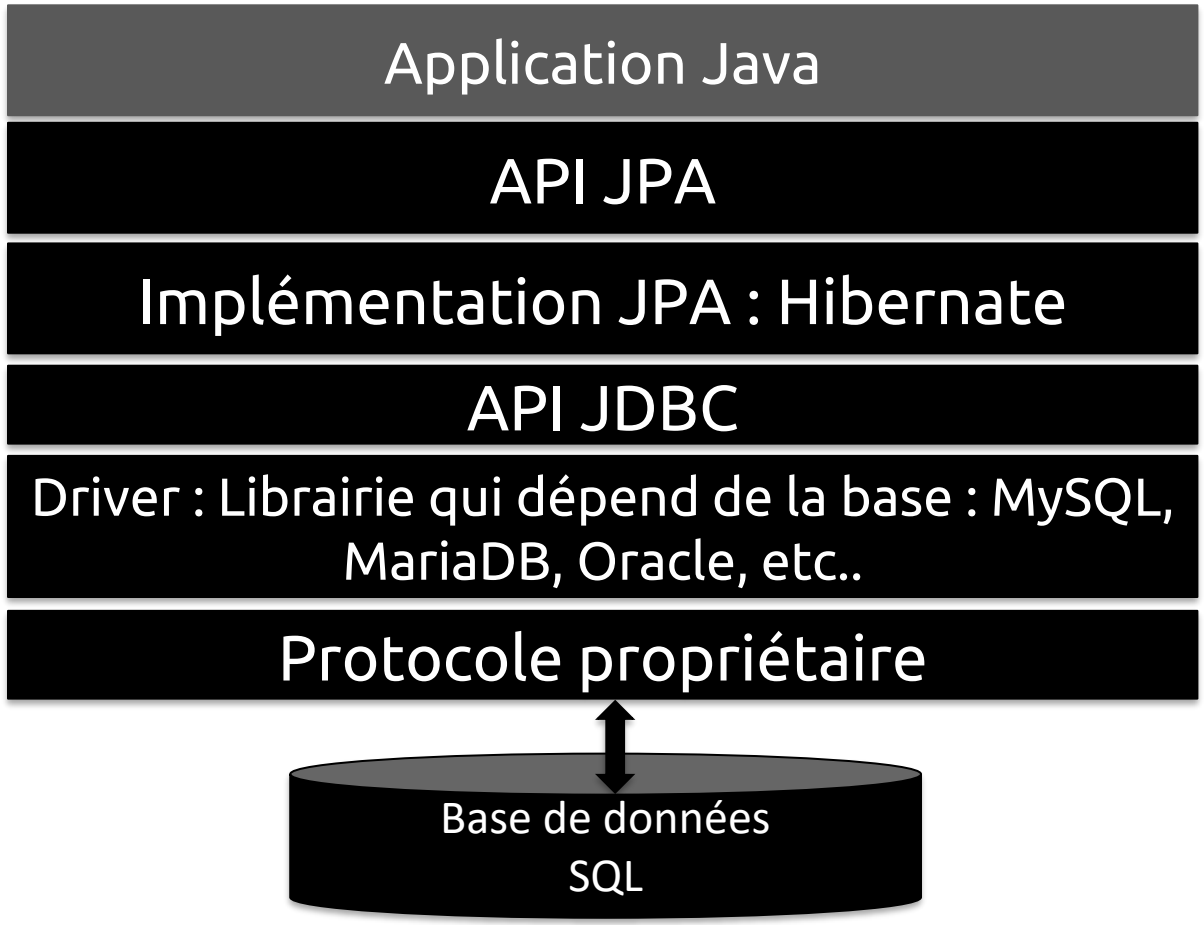
ID	NOM	PRENOM
1	Durand	Paul
2	Dupont	Pierre

JPA = Java Persistence API

- Les entités persistantes sont des POJO (Plain old Java Object)
- Les entités ne se persistent pas elles-mêmes : elles passent par l'**EntityManager**
- L'EntityManager se charge de persister les entités en s'appuyant sur les annotations qu'elles portent
- La classe **Query** encapsule les résultats d'une requête
- Le langage **JPQL** : (Java Persistence Query Language). Une requête JPQL est analogue à une requête SQL sur des objets

Est-ce que JPA suffit pour accéder à une base ?

Non, en plus de l'API JPA, il faut une implémentation de JPA comme Hibernate et aussi la librairie spécifique à la base de données qui contient le **Driver**.



Spring Framework

Introduction à Spring Data JPA

Intro à Spring Data

Spring Data

- Nom “générique” qui désigne beaucoup de sous-projets Spring
- Le but est une **approche cohérente pour l’accès aux données** (relationnelles ou non)
- On va se pencher sur Spring Data JPA

Spring Data

- Spring Data JDBC
- Spring Data JPA
- Spring Data LDAP
- Spring Data MongoDB
- Spring Data Redis
- Spring Data R2DBC
- Spring Data REST
- Spring Data for Apache Cassandra
- Spring Data for Apache Geode
- Spring Data for Apache Solr
- Spring Data for VMware Tanzu GemFire

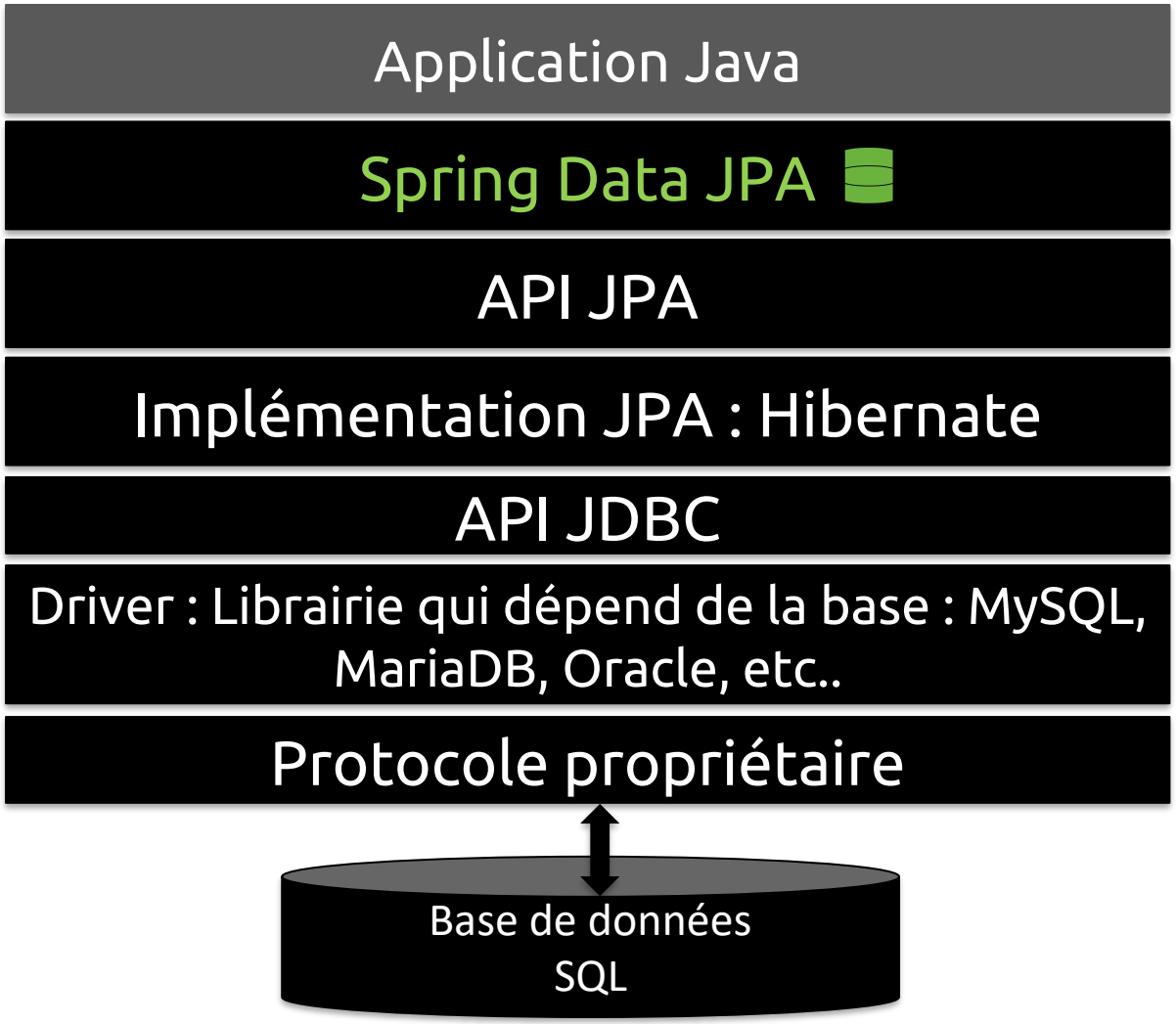
Spring Data JPA

- Facilite l'utilisation de la spécification JPA dans notre projet Spring
- Réduit grandement le *boilerplate code* requis pour implémenter une couche de persistance JPA basique
- Supporte de base la pagination, l'audit, les requêtes dynamiques, QueryDSL, …

Spring Data JPA

- Spring Data JPA **n'est pas un ORM**, c'est une **abstraction** de la couche accès aux données.
- Quand on utilise Spring Data JPA, c'est **Hibernate** est utilisé sous le capot par défaut (voir l'artifact sur mvnrepo)
- Il est bien entendu possible de choisir d'autres implémentations de JPA.

Stack technique



Spring Framework

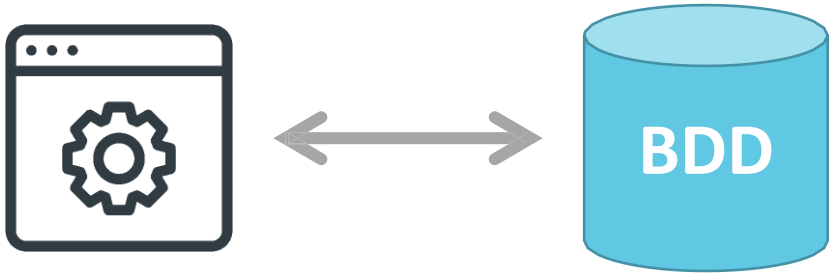
Les Repositories / DAO avec Spring Data JPA

Partie 1 : Utiliser les interfaces Spring

Configurer la datasource dans Spring Boot

Pour configurer notre datasource → **application.properties** :

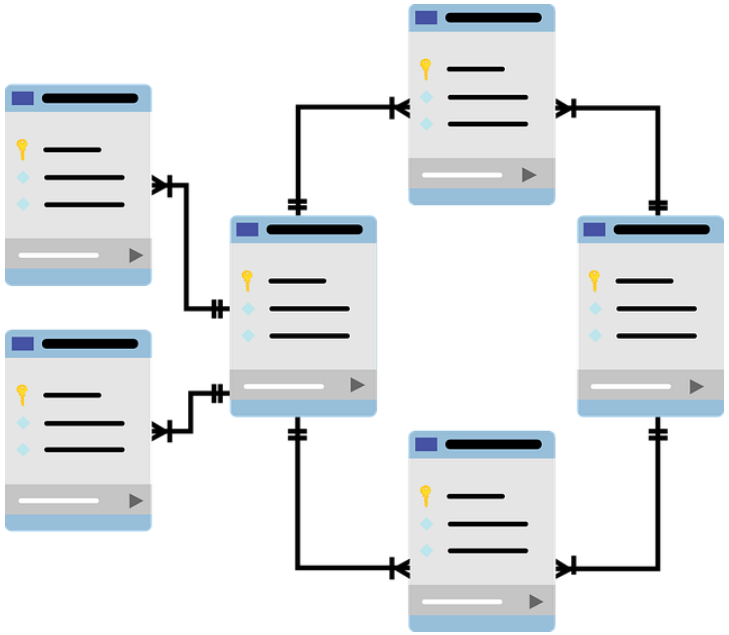
```
# Datasource
spring.datasource.url=jdbc:mysql://localhost:3306/nom-bdd
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate & JPA settings
spring.jpa.hibernate.ddl-auto=update
```



Définir nos entités JPA

L'étape suivante est d'écrire nos entités JPA.

- Nos entités forment la partie **modèle** de notre programme (ou **domain**)



Rappel – Entité JPA

```java
/**
 * Une entité JPA simple
 */
@Entity
@Table(name = "personne_simple")
public class PersonneSimple {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(columnDefinition = "varchar(64)", nullable = false)
    private String nom;

    @Column(columnDefinition = "varchar(64)", nullable = false)
    private String prenom;

    @Enumerated(EnumType.STRING)
    private Genre genre;

    private LocalDateTime dateNaissance;

    private Boolean actif = true;

    // getters & setters -------
}
```

Model

Avec Spring Data JPA :

➜ **rien ne change** concernant l'écriture des entités.

Définir nos “Repositories”

Repository

- Semblable au concept de **DAO** (Data Access Object)
- Leur rôle = exécuter les actions pour **communiquer avec la BDD**
- Classe qui est “au plus proche” de la BDD.
- Chaque entité a son Repository

Définir nos “Repositories”

- Avec Spring Data, un repository est très simple à écrire :

```
@Repository
public interface PersonRepository extends CrudRepository<Person, Long> {
}
```

C’est tout !

- *CrudRepository* est une des **interfaces** qui sont fournies par Spring Data. Il y en a d’autres.
- **Elle nous donne accès à des méthodes génériques CRUD.**

Définir nos “Repositories”

@Repository

```
@Repository
public interface PersonRepository extends CrudRepository<Person, Long> {
}
```

- Annotation **@Repository** sur les classes de type repository/dao.
- Elle fait la même chose que **@Component** (seule la *sémantique* change)
- Une classe avec **@Repository** est donc **un bean Spring.**

Définir nos “Repositories”

extends
(et pas implements)

Classe Typée

```
@Repository
public interface PersonRepository extends CrudRepository<Person, Long> {
}
```

- Pour indiquer la nature de notre Repository, il faut **typer notre interface**
- Avec la classe de l’entité
- Et le type de son ID
- En java une interface peut **extends** une autre interface

Utilisation des repositories

Utilisation du repository avec une injection :

Injection de la dépendance

Par exemple, la méthode findAll

```
@Autowired
private PersonRepository personRepository;

// { ... }

@Override
public void run(String... args) throws Exception {
    System.out.println("Toutes les entités Person : " + this.personRepository.findAll());
}
```

Utilisation des repositories

On utilise une interface directement… et ca marche ?

- C’est là qu’opère la “magie” de Spring.

- Au lancement de notre programme, **Spring implémente automatiquement notre interface** pour en faire une classe normale.

Utilisation des repositories

CrudRepository nous donne par exemple accès aux méthodes suivantes (extrait) :

```
count()                                                    long
toString()                                                 String
delete(Person entity)                                      void
deleteAll()                                                void
deleteAll(Iterable<? extends Person> entities)             void
deleteAllById(Iterable<? extends Integer> ids)             void
deleteById(Integer id)                                     void
existsById(Integer id)                                     boolean
findAll()                                                  Iterable<Person>
findAllById(Iterable<Integer> ids)                         Iterable<Person>
findById(Integer id)                                       Optional<Person>
save(S entity)                                             S
saveAll(Iterable<S> entities)                              Iterable<S>
getClass()                                                 Class<? extends PersonRepository>
notify()                                                   void
notifyAll()                                                void
wait()                                                     void
wait(long timeoutMillis)                                   void
wait(long timeoutMillis, int nanos)                        void
```

Utilisation des repositories – Interfaces

Autres interfaces du même style :

- **PagingAndSortingRepository**
  - Extension de *CrudRepository*
  - Donne accès à des méthodes permettant d'avoir des résultats paginés (Page et Pageable)

- **JpaRepository**
  - Extension de *PagingAndSortingRepository*
  - Fournit des méthodes "query by exemple"

TP 03

Objectifs :

- Mapper des entités JPA sur une BDD existante
- Démonstration des interfaces repository Spring
- Manipuler des données grâce à celles-ci

Spring Framework

Les Repositories / DAO avec Spring Data JPA

Partie 2 : Utiliser les requêtes dérivées du nom de méthode

Repository Spring – Requêtes dérivées du nom

On peut écrire des méthodes qui vont créer une requête selon leur nom :

```
@Repository
public interface PersonRepository extends JpaRepository<Person, Integer> {
    List<Person> findByLastname(String lastname);
}
```

```
this.personRepository.findByLastname("toto")
```

Console

```
Hibernate: select (…) from person person0_ where person0_.lastname=?
```

Repository Spring – Requêtes dérivées du nom

```
List<Person> findByLastname(String lastname);
```

- `List<Person>`
  - Le type de retour indique le **résultat attendu**
  - *Par ex. : List, Set, Page, Iterable, ou une simple entité Person*
- `find(…)By`
  - Type (**sujet**) de la requête, find = un select
  - *Par ex. : find…By, exists…By, count…By, delete…By*
- `Lastname` et `String lastname`
  - **Prédicat** de la requête et son **paramètre**
  - `Lastname` fait référence au **champ de l'entité**

Repository Spring – Requêtes dérivées du nom

```
List<Person> findByLastname(String toto);
```

- Lastname
  - **Fait bien référence au nom de mon champ dans mon entité**

- Le paramètre
  - **Son type est le même que dans mon entité**
  - Son nom n'a pas d'importance

```
@Entity
public class Person {
    @Id
    private Integer id;

    private String lastname;
    private String firstname;
}
```

Repository Spring – Requêtes dérivées du nom

```
@Repository
public interface PersonRepository extends JpaRepository<Person, Integer> {
    // On peut référencer des entités liées sans problème (join automatique)
    List<Person> findByEmailAddressAndLastname(EmailAddress emailAddress, String lastname);

    // Activer le flag Distinct sur la requête
    // « People » est juste là « pour faire joli » (donner du sens)
    List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);
    List<Person> findPeopleDistinctByLastnameOrFirstname(String lastname, String firstname);

    // Activer le IgnoreCase
    List<Person> findByLastnameIgnoreCase(String lastname);
    // Activer le IgnoreCase pour toutes les propriétés éligibles
    List<Person> findByLastnameAndFirstnameAllIgnoreCase(String lastname, String firstname);

    // Activer le ORDER BY sur la requête
    List<Person> findByLastnameOrderByFirstnameAsc(String lastname);
    List<Person> findByLastnameOrderByFirstnameDesc(String lastname);
}
```

Repository Spring – Requêtes dérivées du nom

Prédicats à base de “LIKE” (avec caractères échappés) :

```
List<Person> findByLastnameContainsOrFirstnameContains(String lastname, String firstname);
```

Console

```
Hibernate: select (…) from person person0_ where person0_.lastname like
? escape ? or person0_.firstname like ? escape ?
```

Autres mots-clés :

- *Like, NotLike*
- *StartingWith, StartsWith,*
- *EndingWith, EndsWith,*
- *NotContaining, NotContains, Containing, Contains*

Repository Spring – Requêtes dérivées du nom

Méthode “delete”

```
void deleteByFirstname(String firstname);
```

Console

```
Hibernate: select (…) from person person0_ where person0_.firstname=?
Hibernate: delete from person where id=?
```

- Attention, il faut utiliser l’annotation **@Transactional** sur la méthode qui fait une requête de mise à jour (DML –insert/update/delete/etc)

Repository Spring – Requêtes dérivées du nom

Pour connaître toutes les possibilités de cette fonctionnalité, consulter la doc :

- Query creation
- Liste des *subject keywords*
- Liste des *predicate keywords*
- Liste des types de retours possibles

Repository Spring – Requêtes dérivées du nom

Avantages

- **Facile** et **rapide** pour des requêtes simples
- Beaucoup d'opérateurs et de fonctions utiles (Contains, IgnoreCase, Top10…)

Inconvénients

- Le nom peut devenir **très long**
- Pas toujours utilisable en cas de requête très spécifique

Exemple d'un cas réel :

```
ContratDelegation findByClDeleganteIdAndDelegueeIdAndCommuneDeleganteIdAndContratId(
        Long clDeleganteId,Long delegueeId,Long communeDeleganteId,Long contratId);
```

TP 04

Objectifs :

Écrire des requêtes dérivées du nom de méthode

Spring Framework

Les Repositories / DAO avec Spring Data JPA

Partie 3 : Utiliser @Query

Repository Spring – @Query

- **@Query** pour **déclarer nos requêtes** en JPQL/HQL ou en SQL

JPQL

```
@Query("from Person where firstname = :firstname")
List<Person> findByFirstname(@Param("firstname") String firstname);
```

SQL (nativeQuery = true)

```
@Query(value = "SELECT * FROM person WHERE firstname = :firstname", nativeQuery = true)
List<Person> findByFirstnameSql(@Param("firstname") String firstname);
```

Repository Spring – @Query

JPQL

```
@Query("from Person where firstname = :firstname")
List<Person> findByFirstname(@Param("firstname") String firstname);
```

Remarque en cas de passage de paramètres :

- on utilise **@Param sur l'argument** de la méthode.
- Entre parenthèses on référence le paramètre **par son nom**

→ Doc concernant @Query

Repository Spring – @Query – Requêtes de modification

Il est possible d'écrire des méthodes qui modifient des données :

➜ Il faut rajouter **@Modifying** à notre requête

JPQL : Update

```
@Query("update Person p set p.firstname = :newFirstname where p.firstname = :oldFirstname")
@Modifying
void changePersonWithFirstname(
        @Param("oldFirstname") String oldFirstname,
        @Param("newFirstname") String newFirstname);
```

JPQL : Delete

```
@Query("delete from Person p where p.firstname = :firstname")
@Modifying
void deleteBulkWhereFirstnameEqual(@Param("firstname") String firstname);
```

Repository Spring – @Query – Requêtes de modification

En plus de **@Modifying**,

- il faut parfois rajouter le boolean **clearAutomatically** afin de synchroniser l'EntityManager.

clearAutomatically = true

JPQL : Update

```
@Query("update Person p set p.firstname = :newFirstname where p.firstname = :oldFirstname")
@Modifying(clearAutomatically = true)
void changePersonWithFirstname(
        @Param("oldFirstname") String oldFirstname,
        @Param("newFirstname") String newFirstname);
```

→ Pour en savoir plus (documentation Spring)

TP 05

Objectifs :

Utiliser @Query sur nos repos (DAO)

Spring Framework

Les Repositories / DAO avec Spring Data JPA

Partie 4 : Implémentations personnalisées

Repository Spring – Implémentations personnalisées

Si on doit écrire nous même le code, on peut le faire en 3 étapes.

- Étape 1 : Définir une interface personnalisée

```
public interface PersonRepositoryCustom {
    List<Person> uneMethodePerso();
    List<Person> uneSecondeMethodePerso(Person person);
}
```

- **Pas d'annotation !**
- Y écrire les **signatures** des méthodes que l'on va écrire.
- Le nom n'a pas d'importance mais il vaut mieux garder un nom logique.

Repository Spring – Implémentations personnalisées

- Étape 2 : Implémenter notre interface perso

```
public class PersonRepositoryCustomImpl implements
        PersonRepositoryCustom {

    @PersistenceContext
    private EntityManager em;

    public Person uneMethodePerso() {
        Person p = new Person();
        p.setFirstname("Entity");
        p.setLastname("Manager");
        em.persist(p);
        return p;
    }
}
```

@PersistenceContext pour injecter l'EntityManager

- On peut faire appel à d'autres ***beans*** ou à l'EntityManager.
- **Le nom doit finir par <u>Impl</u>** sinon Spring ne la trouvera pas.

Repository Spring – Implémentations personnalisées

- Étape 3 : Faire étendre notre interface sur le repository Spring

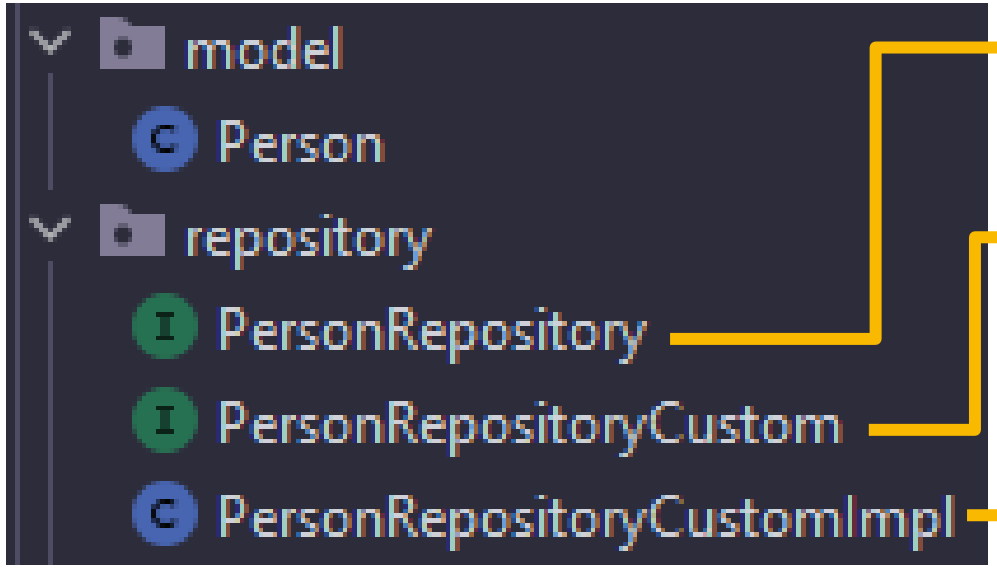
```
public interface PersonRepository
        extends JpaRepository<Person, Integer>, PersonRepositoryCustom {
```

➡ Avec ça, Spring va faire le lien tout seul avec la méthode qu'on a écrite dans la classe qui finit par « *Impl* »

```
    this.personRepository.uneMethodePerso();
```

Repository Spring – Implémentations personnalisées

Au final cela nous donne 1 classe et 2 interfaces :



Le Repository Spring de base

Le Repository « fragment »

Notre classe qui implémente le repository « fragment »

TP 06

Objectifs :

Implémenter notre propre interface et méthode qui utilise l’entity manager

Spring Framework

Les Repositories / DAO avec Spring Data JPA

Partie 5 : Transactions

Repository Spring – Transactions : rappels

Exemple : On effectue plusieurs requêtes successives, chaque opération a une chance de rater.

1.Ajout … … … … ✅
2.Suppression … … ✅
3.Suppression … … ❌
4.Ajout … … … … 🕐

Si 3. rate, on se retrouve avec une base dans un état incohérent.

Intérêt des transactions :

- En cas d'erreur, on fait un **rollback** et on revient à l'état initial
- En cas de réussite, on fait un **commit**, et toute la séquence est réellement validée en base.

Repository Spring – Transactions : rappels

Rappels sur les « Transactions » en BDD (et autres systèmes)

- Elles doivent être **ACID** :
  - ***A**tomicity* : toutes les opérations sont validées ou aucune,
  - ***C**onsistency* : d'un état cohérent à un autre état cohérent,
  - ***I**solation* : les changements sont invisibles hors de cette transaction,
  - ***D**urability* : les changements sont persistés.

- Pour assurer la **viabilité et la cohérence des données.**

Repository Spring – Transactions : rappels

Avec JPA, toute méthode de modification **DML** (insert, update, delete, etc) **requiert d'être dans une transaction !**

Repository Spring – Transactions

- **@Transactional**
  - Sur les méthodes : elles deviennent « transactionnelles »
  - Sur la classe : toutes les méthodes publiques deviennent « transactionnelles »

- Plus besoin de commencer et commit nos transactions à la main.
- Spring va « entourer » la méthode avec le code qu'il faut.

Spring Framework

Tests Unitaires sur les Repositories Spring

Tests Unitaires – Rappels

- Les tests unitaires (TU) testent un élément de façon isolée.
- Si cet élément dépend d'autres éléments pour fonctionner, alors on doit **simuler le fonctionnement** de ces dépendances (*mocks*).
- On instancie cet élément, on teste chaque méthode et on vérifie le résultat attendu.

Tests Unitaires dans Spring Data – Introduction

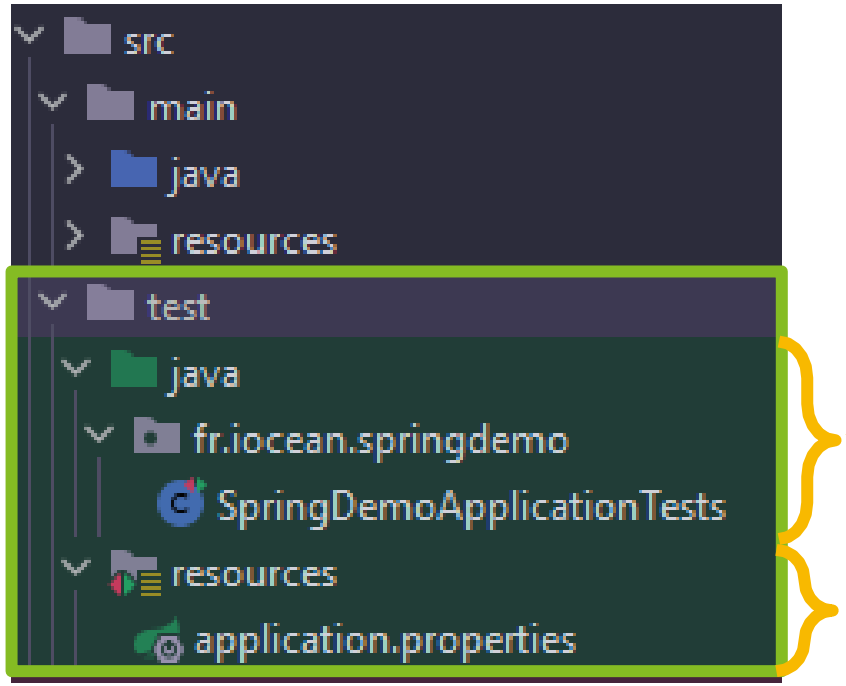
- Spring nous fournit des outils pour écrire des tests unitaires facilement dans un projet Spring.

- Starter « test » :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Tests Unitaires dans Spring Data – Introduction

Dans un projet Maven, toute la partie « tests » se trouve dans un dossier à part



Code source des tests

Dossier *resources* et fichier de config

Tests Unitaires dans Spring Data

- Pour les Tests Unitaires, en général on n'a pas de BDDdédiée.
- Et pas question de tester sur la « vraie » BDD
- **Solution** : utiliser une BDD « en mémoire » comme H2.
- H2 nous permet de ne pas avoir à mettre en place et configurer une BDD classique.

Tests Unitaires dans Spring Data

- Configuration de notre datasource pour les tests unitaires :

src/test/resources/application.properties

```
# Datasource H2 pour les tests unitaires
spring.datasource.url=jdbc:h2:mem:test
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.generate-ddl=true
spring.jpa.hibernate.ddl-auto=create
```

Tests Unitaires dans Spring Data – Exemple

Exemple : on veut écrire le test de cette méthode (tiré du repository Spring)

```
/**
 * Retourne une liste de personnes dont
 *  - le nom contient le paramètre donné "lastname"
 *  - OU le prénom contient le paramètre donné "firstname"
 *
 * @param lastname String que doit contenir le nom
 * @param firstname String que doit contenir le prénom
 * @return une List de Person résultat de la requête
 */
List<Person> findByLastnameContainsOrFirstnameContains(String lastname, String firstname);
```

Tests Unitaires dans Spring Data – Exemple

```
@SpringBootTest
@Transactional
public class PersonRepositoryTest {
    @PersistenceContext
    private EntityManager em;
    @Autowired
    private PersonRepository personRepository;

    @BeforeEach
    private void initData() {
        em.clear();
        Person p2 = new Person();
        p2.setFirstname("James");
        p2.setLastname("Tour");
        em.persist(p2);
        Person p3 = new Person();
        p3.setFirstname("Toto");
        p3.setLastname("Toto");
        em.persist(p3);
        em.flush();
    }

    @Test
    public void findDistinctByLastnameContainsOrFirstnameContainsTest() {
        List<Person> test = this.personRepository.findDistinctByLastnameContainsOrFirstnameContains("To", "To");
        Assertions.assertEquals(2, test.size());
    }
}
```

@SpringBootTest

Notre jeu de données initialisé avec l'em

JUnit classique

Références et Ressources

- Documentation Spring Data
- Query creation
- Liste des *subject keywords*
- Liste des *predicate keywords*
- Liste des types de retours possibles
- @Query